

Central Asia Barometer Data Analysis

Ryan Womack

2026-03-31

Copyright Ryan Womack, 2026. This work is licensed under [CC BY-NC-SA 4.0](#)

Central Asia Barometer Data Analysis

*data wrangling, exploratory data analysis and data visualization of Central Asia Barometer Data
first developed for presentation at [American University of Central Asia](#), Spring 2026*

1 Overview

This workshop illustrates working with data, starting from the basics of importing and setting up data, and moving on to descriptive statistics, basic analysis, and data visualization. Coding is primarily in R, but also with some Python examples. Both R and Python are excellent tools for data analysis, so to some extent your choice is a matter of preference and familiarity. The R language emerged from statistics and added computing functionality, whereas Python is a general purpose computing language to which heavy doses of statistical capabilities have been added.

1.1 Definitions

Statistics is the broad field that provides rigorous tools for data analysis through mathematics. Although we may think of *machine learning* as a computer science topic, the fundamental theory behind machine learning is mostly statistics. Machine learning (ML) refers to many possible methods of using computing on a foundation of mathematical models, in order to understand and make predictions from data. *Artificial intelligence* is also part of machine learning, and until the rise in popularity of *Generative AI*, AI/ML usually referred to a broad suite of analytical techniques. Generative AI (GenAI) refers more narrowly to the AI that makes probabilistic guesses on the basis of massive training data, typically using *Large Language Models* (LLMs). Since GenAI does not provide reproducible, replicable results, it is not as useful for reliable research results. *Vibe coding*, or using GenAI to develop computer code and syntax, may have some applications, but is difficult to use without basic understanding of the programming languages in question. For ongoing projects that require steady modifications to an expanding codebase, GenAI is weaker at creating code that does not alter the structure of the project.

2 Installation

This workshop does *not* cover the installation of R and RStudio, but assumes that you have a working copy available.

R is open source software for statistical analysis, available at [R-project.org](https://www.R-project.org). It is recommended to install R first. R commands can be executed by many editors and IDEs, directly in the terminal, or via script files. However, one of the most powerful and common methods, which we will use here, is to use the RStudio IDE.

RStudio, available from [Posit](https://posit.co/) as an [open source download](#), is a tool for creating and running R code (and Python and a few other languages), with many helpful features and templates for working with the other formats and tools in the R ecosystem. RStudio will make use of your installed version of R, running as a layer on top. Posit is also developing [Positron](#) as a newer IDE with more flexibility to work with R, Python, and other possible languages. This workshop does not use Positron, although the code should also work there.

Both RStudio and Positron are introducing AI suggestion tools built into their interfaces that claim to work with and understand the specific code of your project. These should be preferred to using external services like ChatGPT.

For immediate use without installation, try the following:

- The [Posit Cloud](#) is also a free service that allows you to use RStudio (with some limitations on time and memory) with the creation of an account.

Please see the [Getting Started with R video](#) for more details (or other web tutorials and guides).

[Python](#) is also open-source, and can be used with various editors and tools, which are not detailed here.

2.1 R help

The R help system is easy to access. Use a question mark before the name of a specific function to pull up its manual page (`?sample` or `?rnorm`). Use two question marks to search The help tab in the lower right quadrant of RStudio also allows full access to the built in help, including manuals and searching.

Searching the internet for R help can be useful, since there are a wealth of tutorials, blog posts, and other material on general methods and specific R packages. It is helpful to add something like “R statistics” or “R software” or “R package” to your search to distinguish from all other uses of the letter R!

3 Setup and preparing data

3.1 R packages required

We will use the [pak](#) package for installation as a more complete approach to package management. Replace the `pkg` commands with `install.packages()` versions if you prefer.

This session relies on the *tidyverse* suite of packages for data manipulation as a preference, although the same tasks could be accomplished in base R using other functions. Install *pak* and *tidyverse* if you don't already have them on your system. We will also need *reticulate* to run Python code chunks.

```
install.packages("pak", dependencies=TRUE)
library(pak)
pkg_install("tidyverse")
pkg_install("readxl")
pkg_install("reticulate")
devtools::session_info()
```

Note that the `update.packages()` command can be run periodically to check for newer versions of any packages installed.

Load the packages so that we can work with them.

```
library(readxl)
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.0      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.2      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.1
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

3.2 Versions

The links to the right provide alternate modes of accessing this content. You can choose Light or Dark mode HTML, a downloadable PDF, or a Word Docx format file all representing the same content. Code files with R (.R) or Python (.py) are also available, if you'd like to isolate the code from the description.

3.3 Kyrgyz and Russian language

Also, machine-translated versions of this content in Russian and Kyrgyz are linked on the right.

3.4 Python support

Now let's load *reticulate* (for Python support only). We must set the correct location of python corresponding to our system.

A Python file is provided for those who want to try out the same data steps in Python. Note that the Python file was machine-translated from R.

```
# Sys.setenv(RETICULATE_PYTHON = "/usr/bin/python3")
library(reticulate)
use_python("/usr/bin/python3")
```

4 Central Asia Barometer

As you are probably aware, the [Central Asia Barometer](#) project conducts polls and surveys across the countries of Central Asia (Kazakhstan, Kyrgyzstan, Tajikistan, Turkmenistan, and Uzbekistan) and releases the results on a regular basis. Except for the latest wave of data, the CA Barometer data is freely downloadable from their website after providing a contact e-mail. These surveys examine contemporary political and social attitudes, and allow for longitudinal comparisons over time and cross-country comparisons.

For this initial workshop, we will get familiar with the CA Barometer by looking at recent data (2022) for one country (Kyrgyzstan). The second workshop will extend and deepen our understanding by adding additional years and countries to our analysis, and explore more complex data manipulation and analysis.

4.1 Understanding the data

Before worrying about coding or statistical analysis, the first step in working with real-world data is to develop an understanding of its structure and meaning. We want to know what data has been collected, how it has been organized, and think about what kinds of questions it can answer. Only then can we know how to properly proceed with analyzing it.

For CA Barometer, this means looking at the [Questionnaire \(for the 2022 wave\)](#) and the [Methods Report](#) for that year.

5 Download and import data

We download data from [Central Asia Barometer](#) at <https://ca-barometer.org/assets/files/projects/cab-survey-wave-12-excel-kyrgyzstan-2022-autumn.xlsx>

```
download.file("https://ca-barometer.org/assets/files/projects/cab-survey-wave-12-excel-kyrgyzstan-2022-autumn.xlsx",
  "CAB_2022.csv")
mydata <- read_excel("CAB_2022.csv")
mydata
```

```
# A tibble: 1,518 x 158
      MM1 MM2b MM3a MM3b MM4      MM5 MM6      MM7 MM8
    <dbl> <dbl> <dbl> <chr> <chr>    <dbl> <chr>    <dbl> <dtm>
1 100017 778     6 CATI  Kyrgyzstan 329 Female 1 2022-11-01 00:00:00
2 100028 771     6 CATI  Kyrgyzstan 163 Female 1 2022-11-01 00:00:00
3 100035 505     6 CATI  Kyrgyzstan 329 Female 1 2022-11-01 00:00:00
4 100036 509     6 CATI  Kyrgyzstan 311 Female 1 2022-11-01 00:00:00
5 100048 706     6 CATI  Kyrgyzstan 329 Female 1 2022-11-01 00:00:00
6 100054 504     6 CATI  Kyrgyzstan 329 Female 1 2022-11-01 00:00:00
7 100085 502     6 CATI  Kyrgyzstan 210 Female 1 2022-11-01 00:00:00
8 100086 708     6 CATI  Kyrgyzstan 251 Female 1 2022-11-01 00:00:00
9 100087 500     6 CATI  Kyrgyzstan 329 Female 1 2022-11-01 00:00:00
10 100099 553     6 CATI  Kyrgyzstan 329 Female 1 2022-11-01 00:00:00
# i 1,508 more rows
# i 149 more variables: MM9 <dtm>, MM10 <chr>, MM11 <chr>, MM12 <dtm>,
# MM13 <dtm>, MM14 <chr>, A1a <chr>, A1b <chr>, A2 <chr>, A3a <chr>,
# A3b <chr>, A4a <chr>, A4b <chr>, A4c <chr>, A4d <chr>, A4e <chr>,
# A4f <chr>, A5a <chr>, A5b <chr>, A5c <chr>, A5d <chr>, A6 <chr>, A7 <chr>,
# B1 <chr>, B2 <chr>, B3 <chr>, B4a <chr>, B4b <chr>, B5a <chr>, B5b <chr>,
# B6a <chr>, B6b <chr>, B6c <chr>, B6d <chr>, B6e <chr>, B7 <chr>, ...
```

```
attach(mydata)
```

6 Data Wrangling

6.1 Selection and naming

We select interesting variables from the questionnaire to analyze. We'll also change their names to something more memorable.

We want to study across several dimensions:

- language, region, income, sex

and relate this to opinions. We can spend some time looking at the questionnaire to see how these variables are labeled.

Note that MM4 is the country code, but we have already loaded the Kyrgyz file, so we don't need to explicitly select this. Also note that ethnic group is already labeled 'EthnicGrp'. Other variables we will make more intuitive to access by renaming them.

```
names(mydata)
```

```
[1] "MM1"      "MM2b"     "MM3a"     "MM3b"     "MM4"      "MM5"
[7] "MM6"      "MM7"      "MM8"      "MM9"      "MM10"     "MM11"
[13] "MM12"     "MM13"     "MM14"     "A1a"      "A1b"      "A2"
[19] "A3a"      "A3b"      "A4a"      "A4b"      "A4c"      "A4d"
[25] "A4e"      "A4f"      "A5a"      "A5b"      "A5c"      "A5d"
[31] "A6"       "A7"       "B1"       "B2"       "B3"       "B4a"
```

```

[37] "B4b"      "B5a"      "B5b"      "B6a"      "B6b"      "B6c"
[43] "B6d"      "B6e"      "B7"       "B8"       "B9a"      "B9b"
[49] "B9c"      "B9d"      "B10"      "B11a"     "B11b"     "B12"
[55] "B13"      "B14"      "B15a"     "B15b"     "B15c"     "B15d"
[61] "B16"      "D1"       "D2a"      "D2b"      "E1a"      "E1b"
[67] "E1c"      "E1d"      "E1e"      "E2a"      "E2b"      "E2c"
[73] "E2d"      "E2e"      "E3"       "E4a"      "E4b"      "E5a"
[79] "E5b"      "E5c"      "E6a"      "E6b"      "E7a"      "E7b"
[85] "E7c"      "E8a"      "E8b"      "E8c"      "E8d"      "E8e"
[91] "E9a"      "E9b"      "E10"      "E11"      "E12"      "E13"
[97] "E14"      "E15a"     "E15b"     "E15c"     "E16"      "E17"
[103] "E19a"     "E19b"     "E19c"     "E20a"     "E20b"     "E20c"
[109] "E21"      "G1a"      "G1b"      "G2"       "G3a"      "G3b"
[115] "G4"       "G5"       "G6"       "G7"       "G8"       "G9"
[121] "H1"       "H2"       "H3"       "H4"       "H5a"      "H5b"
[127] "DD1"      "DD2"      "DD3"      "DD4"      "DD5"      "DD6a"
[133] "DD6b"     "DD6c"     "DD7a"     "DD7b"     "DD7c"     "DD8"
[139] "DD9a"     "DD9b"     "DD9c"     "DD9d"     "DD10"     "DD11"
[145] "DD12"     "DD13"     "DD14"     "DD15"     "DD16"     "DD17"
[151] "DD18a"    "DD18b"    "DD19"     "DD20"     "EthnicGrp" "AgeGrp"
[157] "FinalWgt1" "FinalWgt2"

```

```

mydata <- mydata |> rename('Sex'='DD1', 'Region'='MM10',
  ↳ 'Language'='DD11', 'Income'='DD8')
attach(mydata)

```

The following objects are masked from mydata (pos = 3):

```

A1a, A1b, A2, A3a, A3b, A4a, A4b, A4c, A4d, A4e, A4f, A5a, A5b,
A5c, A5d, A6, A7, AgeGrp, B1, B10, B11a, B11b, B12, B13, B14, B15a,
B15b, B15c, B15d, B16, B2, B3, B4a, B4b, B5a, B5b, B6a, B6b, B6c,
B6d, B6e, B7, B8, B9a, B9b, B9c, B9d, D1, D2a, D2b, DD10, DD12,
DD13, DD14, DD15, DD16, DD17, DD18a, DD18b, DD19, DD2, DD20, DD3,
DD4, DD5, DD6a, DD6b, DD6c, DD7a, DD7b, DD7c, DD9a, DD9b, DD9c,
DD9d, E10, E11, E12, E13, E14, E15a, E15b, E15c, E16, E17, E19a,
E19b, E19c, E1a, E1b, E1c, E1d, E1e, E20a, E20b, E20c, E21, E2a,
E2b, E2c, E2d, E2e, E3, E4a, E4b, E5a, E5b, E5c, E6a, E6b, E7a,
E7b, E7c, E8a, E8b, E8c, E8d, E8e, E9a, E9b, EthnicGrp, FinalWgt1,
FinalWgt2, G1a, G1b, G2, G3a, G3b, G4, G5, G6, G7, G8, G9, H1, H2,
H3, H4, H5a, H5b, MM1, MM11, MM12, MM13, MM14, MM2b, MM3a, MM3b,
MM4, MM5, MM6, MM7, MM8, MM9

```

6.2 Descriptive statistics

Let's take a look at several of these variables and see how they're distributed. A simple *table* command will do the trick for getting initial familiarity with the data. While some large datasets are difficult to gain intuitive understanding of, for this kind of social science data, nothing beats

spending time looking over the data before getting into more complex analysis. We may also discover outliers or issues to remedy before moving forward.

`table(Sex)`

```
Sex
Female   Male
    750    768
```

`table(Region)`

```
Region
      Batkenskaja      Bishkek      Chujskaja Dzhalal-Abadskaja
          105          399          228          222
Issyk-Kul'skaja      Narynskaja      Osh City      Oshskaja
          142          83          65          214
      Talaskaja
          60
```

`table(Language)`

```
Language
Don't Know (vol.)      Kazakh      Kyrgyz      Other (vol.)
          2              1          1280          14
      Refused (vol.)      Russian      Tajik      Turkmen
          1             154           1          2
          Uzbek
          63
```

`table(Income)`

```
Income
10,001 - 12,000 som  12,001 - 16,000 som  16,001 - 20,000 som
          71          136          160
      2,801 - 4,100 som  20,001 - 28,000 som  4,101 - 6,000 som
          33          185          46
      6,001 - 8,000 som  8,001 - 10,000 som  Don't Know (vol.)
          36          96          84
Less than 2,801 som  More than 28,000 som  Refused (vol.)
          40          570          61
```

`table(EthnicGrp)`

```
EthnicGrp
Kyrgyz   Other Russian   Uzbek
    1315         62         63         78
```

`table(Sex, Language)`

Sex	Language					
	Don't Know (vol.)	Kazakh	Kyrgyz	Other (vol.)	Refused (vol.)	Russian
Female	1	0	627	5	1	92
Male	1	1	653	9	0	62

Sex	Language		
	Tajik	Turkmen	Uzbek
Female	1	1	22
Male	0	1	41

`table(Income, Sex)`

Income	Sex	
	Female	Male
10,001 - 12,000 som	36	35
12,001 - 16,000 som	71	65
16,001 - 20,000 som	84	76
2,801 - 4,100 som	17	16
20,001 - 28,000 som	95	90
4,101 - 6,000 som	30	16
6,001 - 8,000 som	21	15
8,001 - 10,000 som	56	40
Don't Know (vol.)	49	35
Less than 2,801 som	17	23
More than 28,000 som	251	319
Refused (vol.)	23	38

`table(E17)`

E17

A few more months	A few more weeks	About another week	Another month
333	21	27	65
Another year	Don't know (vol.)	More than a year	Refused (vol.)
279	373	209	16
Several more days	Two years or more		
32	163		

6.3 Recoding data

One issue that we discover is that Income is reported in categories, by range. If we want to treat Income as a number, we will have to recode it. We set each range to its rough midpoint in the code below, using the `recode_values` command.

```
mydata$Income_Numeric <- recode_values(
  Income,
  "Less than 2,801 som" ~ 1400,
  "2,801 - 4,100 som" ~ 3400,
  "4,101 - 6,000 som" ~ 5000,
  "6,001 - 8,000 som" ~ 7000,
```



```

"8,001 - 10,000 som" ~ 9000,
"10,001 - 12,000 som" ~ 11000,
"12,001 - 16,000 som" ~ 14000,
"16,001 - 20,000 som" ~ 18000,
"20,001 - 28,000 som" ~ 24000,
"More than 28,000 som" ~ 40000,
c("Don't Know (vol.)", "Refused (vol.)") ~ NA
)

```

We reattach the data to make sure the new variable, Income_Numeric, is easily available, then check how it turned out.

```
attach(mydata)
```

The following objects are masked from mydata (pos = 3):

```

A1a, A1b, A2, A3a, A3b, A4a, A4b, A4c, A4d, A4e, A4f, A5a, A5b,
A5c, A5d, A6, A7, AgeGrp, B1, B10, B11a, B11b, B12, B13, B14, B15a,
B15b, B15c, B15d, B16, B2, B3, B4a, B4b, B5a, B5b, B6a, B6b, B6c,
B6d, B6e, B7, B8, B9a, B9b, B9c, B9d, D1, D2a, D2b, DD10, DD12,
DD13, DD14, DD15, DD16, DD17, DD18a, DD18b, DD19, DD2, DD20, DD3,
DD4, DD5, DD6a, DD6b, DD6c, DD7a, DD7b, DD7c, DD9a, DD9b, DD9c,
DD9d, E10, E11, E12, E13, E14, E15a, E15b, E15c, E16, E17, E19a,
E19b, E19c, E1a, E1b, E1c, E1d, E1e, E20a, E20b, E20c, E21, E2a,
E2b, E2c, E2d, E2e, E3, E4a, E4b, E5a, E5b, E5c, E6a, E6b, E7a,
E7b, E7c, E8a, E8b, E8c, E8d, E8e, E9a, E9b, EthnicGrp, FinalWgt1,
FinalWgt2, G1a, G1b, G2, G3a, G3b, G4, G5, G6, G7, G8, G9, H1, H2,
H3, H4, H5a, H5b, Income, Language, MM1, MM11, MM12, MM13, MM14,
MM2b, MM3a, MM3b, MM4, MM5, MM6, MM7, MM8, MM9, Region, Sex

```

The following objects are masked from mydata (pos = 4):

```

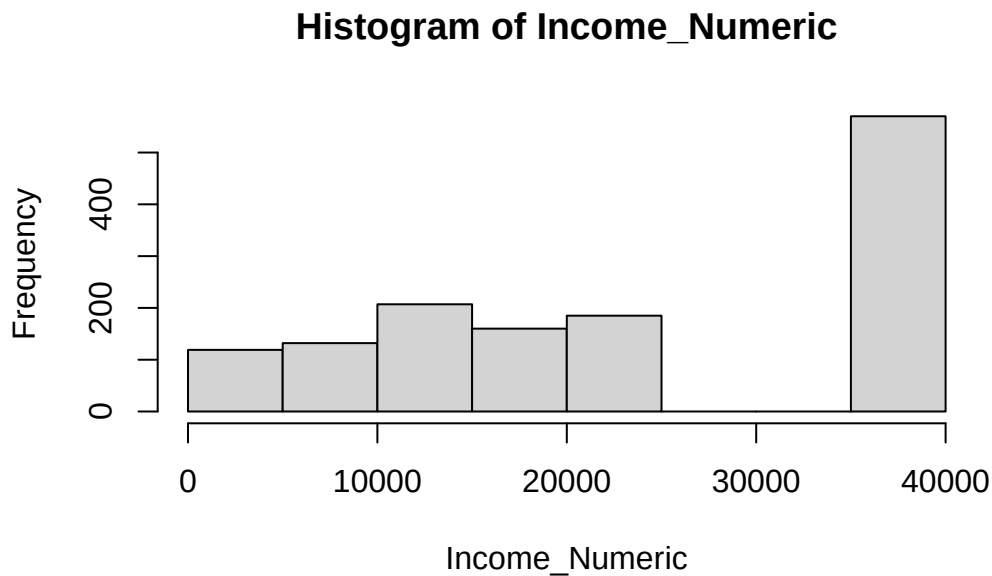
A1a, A1b, A2, A3a, A3b, A4a, A4b, A4c, A4d, A4e, A4f, A5a, A5b,
A5c, A5d, A6, A7, AgeGrp, B1, B10, B11a, B11b, B12, B13, B14, B15a,
B15b, B15c, B15d, B16, B2, B3, B4a, B4b, B5a, B5b, B6a, B6b, B6c,
B6d, B6e, B7, B8, B9a, B9b, B9c, B9d, D1, D2a, D2b, DD10, DD12,
DD13, DD14, DD15, DD16, DD17, DD18a, DD18b, DD19, DD2, DD20, DD3,
DD4, DD5, DD6a, DD6b, DD6c, DD7a, DD7b, DD7c, DD9a, DD9b, DD9c,
DD9d, E10, E11, E12, E13, E14, E15a, E15b, E15c, E16, E17, E19a,
E19b, E19c, E1a, E1b, E1c, E1d, E1e, E20a, E20b, E20c, E21, E2a,
E2b, E2c, E2d, E2e, E3, E4a, E4b, E5a, E5b, E5c, E6a, E6b, E7a,
E7b, E7c, E8a, E8b, E8c, E8d, E8e, E9a, E9b, EthnicGrp, FinalWgt1,
FinalWgt2, G1a, G1b, G2, G3a, G3b, G4, G5, G6, G7, G8, G9, H1, H2,
H3, H4, H5a, H5b, MM1, MM11, MM12, MM13, MM14, MM2b, MM3a, MM3b,
MM4, MM5, MM6, MM7, MM8, MM9

```

```
mean(Income_Numeric, na.rm=TRUE)
```

```
[1] 24995.78
```

```
hist(Income_Numeric)
```



```
mydata |>
group_by(Sex) |>
summarize(Inc = mean(Income_Numeric, na.rm=TRUE))
```

```
# A tibble: 2 x 2
  Sex      Inc
<chr> <dbl>
1 Female 23753.
2 Male   26208.
```

```
# t-tests or chi-sq for significance
```

```
# try a t-test
```

```
# is income significantly different by sex - yes
```

```
t.test(Income_Numeric~Sex)
```

Welch Two Sample t-test

```
data: Income_Numeric by Sex
```

```
t = -3.3228, df = 1370.6, p-value = 0.0009147
```

```
alternative hypothesis: true difference in means between group Female and group Male is not eq
```

```
95 percent confidence interval:
```

```
-3904.313 -1005.607
```

```
sample estimates:
```

```
mean in group Female    mean in group Male
      23753.10          26208.06
```

7 Plots - Data Visualization

We can pick any general issue to focus on, such as which messenger app people use. Actually, in Kyrgyzstan everyone uses WhatsApp, so perhaps not so interesting! Let's pick something more general, like whether the country is going in the right or wrong direction.

`table(Sex, A3a)`

A3a				
Sex	Do not use chat rooms or messengers	Don't Know (vol.)	Other (vol.)	
Female		16	2	1
Male		22	1	7

A3a				
Sex	Telegram	Viber	WhatsApp	
Female	41	0	690	
Male	52	1	685	

`table(D1)`

D1				
Don't Know (vol.)	Refused (vol.)	Right direction	Wrong direction	
138	16	1157	207	

`table(D1, Sex)`

Sex			
D1	Female	Male	
Don't Know (vol.)	72	66	
Refused (vol.)	9	7	
Right direction	579	578	
Wrong direction	90	117	

`table(D1, Region)`

Region					
D1	Batkinskaja	Bishkek	Chujskaja	Dzhalal-Abadskaja	
Don't Know (vol.)	5	40	22	19	
Refused (vol.)	1	2	5	3	
Right direction	90	266	162	181	
Wrong direction	9	91	39	19	

Region						
D1	Issyk-Kul'skaja	Narynskaja	Osh City	Oshskaja	Talaskaja	
Don't Know (vol.)	6	8	8	18	12	
Refused (vol.)	3	0	0	2	0	
Right direction	125	69	53	173	38	
Wrong direction	8	6	4	21	10	

`table(D1, Language)`

	Language				
D1	Don't Know (vol.)	Kazakh	Kyrgyz	Other (vol.)	Refused (vol.)
Don't Know (vol.)	1	0	112	1	0
Refused (vol.)	0	0	10	0	1
Right direction	1	1	1009	11	0
Wrong direction	0	0	149	2	0

	Language			
D1	Russian	Tajik	Turkmen	Uzbek
Don't Know (vol.)	20	0	0	4
Refused (vol.)	4	0	0	1
Right direction	79	1	2	53
Wrong direction	51	0	0	5

```
table(D1, Income_Numeric)
```

	Income_Numeric									
D1	1400	3400	5000	7000	9000	11000	14000	18000	24000	40000
Don't Know (vol.)	3	1	3	2	11	6	7	17	15	50
Refused (vol.)	0	2	0	0	1	1	2	0	1	7
Right direction	31	26	38	30	78	56	115	125	147	414
Wrong direction	6	4	5	4	6	8	12	18	22	99

Recode to drop 'Don't Know' and 'Refused'

```
mydata$D1_recode <- recode_values(
  D1,
  "Right direction" ~ "Right",
  "Wrong direction" ~ "Wrong",
  c("Don't Know (vol.)", "Refused (vol.)") ~ NA
)

mydata$D1_recode <- as.factor(mydata$D1_recode)
```

```
attach(mydata)
```

The following objects are masked from mydata (pos = 3):

A1a, A1b, A2, A3a, A3b, A4a, A4b, A4c, A4d, A4e, A4f, A5a, A5b, A5c, A5d, A6, A7, AgeGrp, B1, B10, B11a, B11b, B12, B13, B14, B15a, B15b, B15c, B15d, B16, B2, B3, B4a, B4b, B5a, B5b, B6a, B6b, B6c, B6d, B6e, B7, B8, B9a, B9b, B9c, B9d, D1, D2a, D2b, DD10, DD12, DD13, DD14, DD15, DD16, DD17, DD18a, DD18b, DD19, DD2, DD20, DD3, DD4, DD5, DD6a, DD6b, DD6c, DD7a, DD7b, DD7c, DD9a, DD9b, DD9c, DD9d, E10, E11, E12, E13, E14, E15a, E15b, E15c, E16, E17, E19a, E19b, E19c, E1a, E1b, E1c, E1d, E1e, E20a, E20b, E20c, E21, E2a, E2b, E2c, E2d, E2e, E3, E4a, E4b, E5a, E5b, E5c, E6a, E6b, E7a, E7b, E7c, E8a, E8b, E8c, E8d, E8e, E9a, E9b, EthnicGrp, FinalWgt1, FinalWgt2, G1a, G1b, G2, G3a, G3b, G4, G5, G6, G7, G8, G9, H1, H2,

H3, H4, H5a, H5b, Income, Income_Numeric, Language, MM1, MM11, MM12, MM13, MM14, MM2b, MM3a, MM3b, MM4, MM5, MM6, MM7, MM8, MM9, Region, Sex

The following objects are masked from mydata (pos = 4):

A1a, A1b, A2, A3a, A3b, A4a, A4b, A4c, A4d, A4e, A4f, A5a, A5b, A5c, A5d, A6, A7, AgeGrp, B1, B10, B11a, B11b, B12, B13, B14, B15a, B15b, B15c, B15d, B16, B2, B3, B4a, B4b, B5a, B5b, B6a, B6b, B6c, B6d, B6e, B7, B8, B9a, B9b, B9c, B9d, D1, D2a, D2b, DD10, DD12, DD13, DD14, DD15, DD16, DD17, DD18a, DD18b, DD19, DD2, DD20, DD3, DD4, DD5, DD6a, DD6b, DD6c, DD7a, DD7b, DD7c, DD9a, DD9b, DD9c, DD9d, E10, E11, E12, E13, E14, E15a, E15b, E15c, E16, E17, E19a, E19b, E19c, E1a, E1b, E1c, E1d, E1e, E20a, E20b, E20c, E21, E2a, E2b, E2c, E2d, E2e, E3, E4a, E4b, E5a, E5b, E5c, E6a, E6b, E7a, E7b, E7c, E8a, E8b, E8c, E8d, E8e, E9a, E9b, EthnicGrp, FinalWgt1, FinalWgt2, G1a, G1b, G2, G3a, G3b, G4, G5, G6, G7, G8, G9, H1, H2, H3, H4, H5a, H5b, Income, Language, MM1, MM11, MM12, MM13, MM14, MM2b, MM3a, MM3b, MM4, MM5, MM6, MM7, MM8, MM9, Region, Sex

The following objects are masked from mydata (pos = 5):

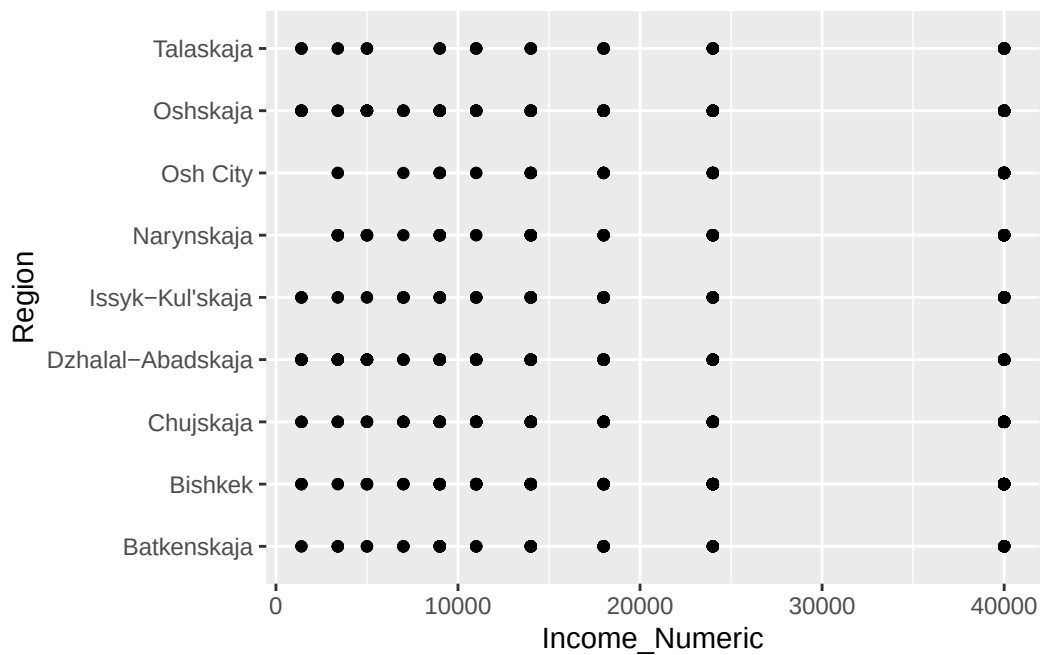
A1a, A1b, A2, A3a, A3b, A4a, A4b, A4c, A4d, A4e, A4f, A5a, A5b, A5c, A5d, A6, A7, AgeGrp, B1, B10, B11a, B11b, B12, B13, B14, B15a, B15b, B15c, B15d, B16, B2, B3, B4a, B4b, B5a, B5b, B6a, B6b, B6c, B6d, B6e, B7, B8, B9a, B9b, B9c, B9d, D1, D2a, D2b, DD10, DD12, DD13, DD14, DD15, DD16, DD17, DD18a, DD18b, DD19, DD2, DD20, DD3, DD4, DD5, DD6a, DD6b, DD6c, DD7a, DD7b, DD7c, DD9a, DD9b, DD9c, DD9d, E10, E11, E12, E13, E14, E15a, E15b, E15c, E16, E17, E19a, E19b, E19c, E1a, E1b, E1c, E1d, E1e, E20a, E20b, E20c, E21, E2a, E2b, E2c, E2d, E2e, E3, E4a, E4b, E5a, E5b, E5c, E6a, E6b, E7a, E7b, E7c, E8a, E8b, E8c, E8d, E8e, E9a, E9b, EthnicGrp, FinalWgt1, FinalWgt2, G1a, G1b, G2, G3a, G3b, G4, G5, G6, G7, G8, G9, H1, H2, H3, H4, H5a, H5b, MM1, MM11, MM12, MM13, MM14, MM2b, MM3a, MM3b, MM4, MM5, MM6, MM7, MM8, MM9

7.1 Illustrative plots

Here are a few illustrative plots. We're not trying to make them pretty at this point, just trying to quickly understand our data.

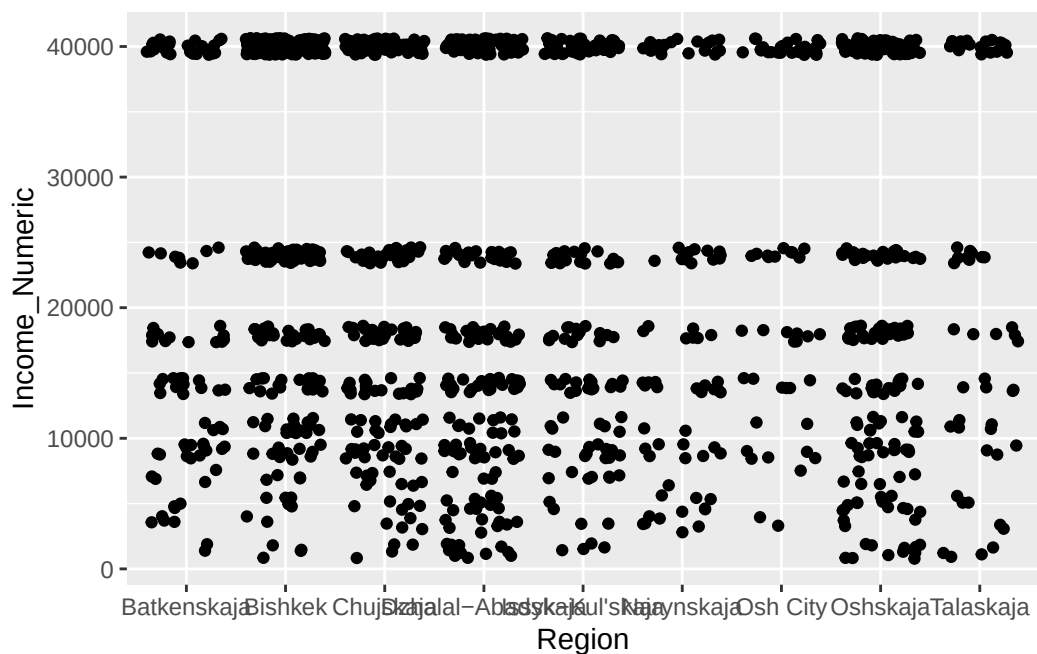
```
# Some illustrative plots
ggplot(data=mydata, aes(x=Income_Numeric, y=Region))+geom_point()
```

Warning: Removed 145 rows containing missing values or values outside the scale range (`geom\_point()`).

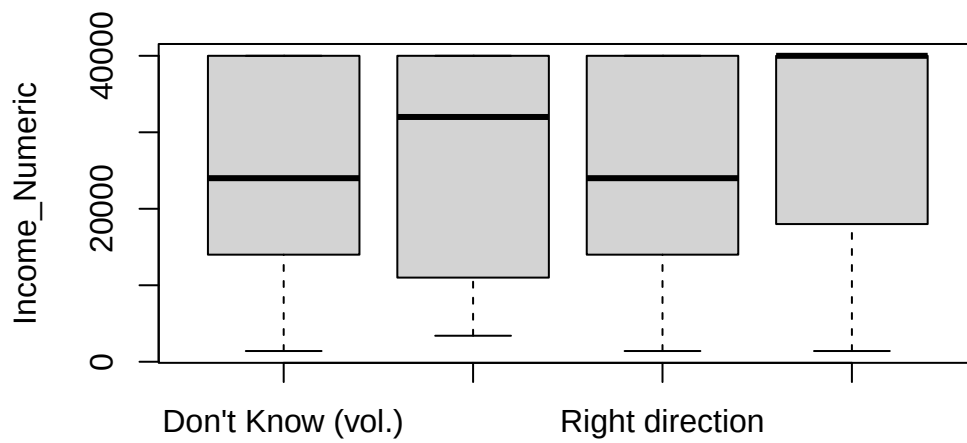


```
ggplot(data=mydata, aes(x=Region, y=Income_Numeric))+geom_jitter()
```

Warning: Removed 145 rows containing missing values or values outside the scale range (`geom\_point()`).

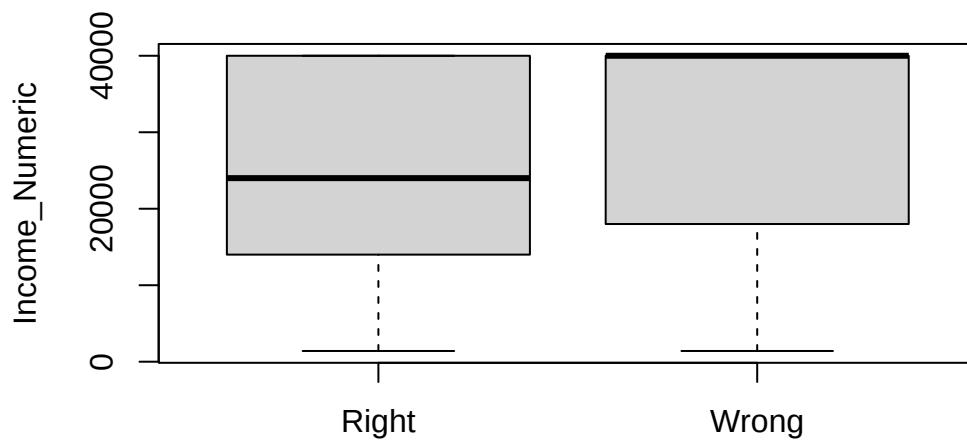


```
boxplot(Income_Numeric~D1)
```



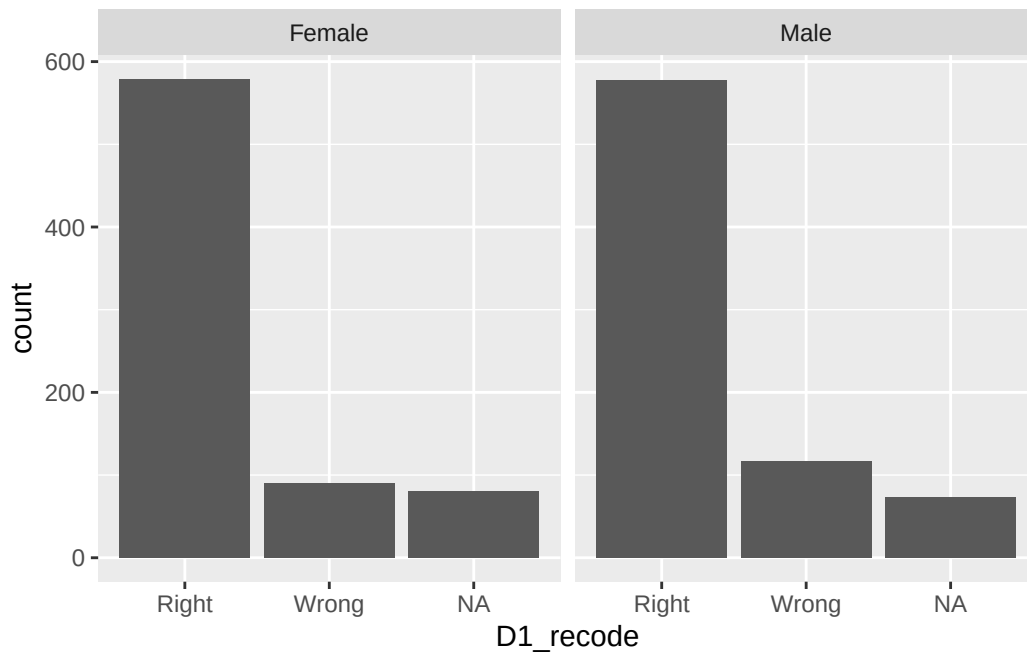
D1

`boxplot(Income_Numeric~D1_recode)`



D1_recode

`ggplot(mydata, aes(x=D1_recode))+facet_grid(~Sex)+geom_bar()`



8 Further statistical analysis

8.1 Logistic regression

As an example of statistical analysis, we can try a logistic regression to predict the answer to the right direction question. Logistic regression is the suitable method for when the outcome is a binary choice (yes/no, true/false, right/wrong).

```
logistic_output <- glm(D1_recode ~ Sex + Language + Income_Numeric,
  ↪ family=binomial, data=mydata)
summary(logistic_output)
```

Call:

```
glm(formula = D1_recode ~ Sex + Language + Income_Numeric, family = binomial,
  data = mydata)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-1.504e+01	8.827e+02	-0.017	0.98641
SexMale	3.881e-01	1.688e-01	2.298	0.02154 *
LanguageKazakh	-2.200e-01	1.248e+03	0.000	0.99986
LanguageKyrgyz	1.246e+01	8.827e+02	0.014	0.98874
LanguageOther (vol.)	1.258e+01	8.827e+02	0.014	0.98863
LanguageRussian	1.390e+01	8.827e+02	0.016	0.98744
LanguageTajik	6.654e-02	1.248e+03	0.000	0.99996
LanguageTurkmen	-4.101e-01	1.080e+03	0.000	0.99970
LanguageUzbek	1.178e+01	8.827e+02	0.013	0.98935


```
Income_Numeric          1.692e-05  6.179e-06   2.739  0.00617 **
```

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

(Dispersion parameter for binomial family taken to be 1)

```
Null deviance: 1042.64  on 1243  degrees of freedom
Residual deviance:  983.01  on 1234  degrees of freedom
(274 observations deleted due to missingness)
AIC: 1003
```

Number of Fisher Scoring iterations: 13

9 Other possibilities

These are some notes on additional interesting variables in the data, although feel free to pick your own and explore.

Question B13 - *“Have Russians been emigrating to your area?”* could be analyzed by region

Question E17 - *“How long will conflict with Ukraine last?”* asked in 2022

Question G3b - *“Attitude toward President Japarov?”* what breakdowns would be interesting here?

10 Conclusion

There is no one stopping point for data analysis. Keep going until you have a full understanding of the data and its interaction with the questions you seek to answer. The second part of this workshop series will illustrate more advanced methods and combinations of data.

See [Data Analysis 1](#) and [Data Analysis 2](#) for further discussion of data analysis techniques.

10.1 R for Data Science

Consult the [R for Data Science book](#) to learn more about data wrangling. Also check out [tidy-models](#) a collection of packages for modeling and machine learning using tidyverse principles.